

# CYnchronize

Base de données Oracle multi-sites Cergy / Pau

Rapport de projet

Younes Douici   Abdelah El Harsal   Adam Terrak   Éléonore Georgel

Traitement et Administration des Données ING 2 GIA 1

CY Tech, année 2025-2026

17 mai 2026

Dépôt : [github.com/NayJi7/CYnchronize](https://github.com/NayJi7/CYnchronize)

## Table des matières

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                | <b>3</b>  |
| 1.1      | Contexte . . . . .                                                 | 3         |
| 1.2      | Choix du périmètre fonctionnel . . . . .                           | 3         |
| 1.3      | Architecture globale . . . . .                                     | 3         |
| 1.4      | Stack technique . . . . .                                          | 4         |
| <b>2</b> | <b>Reverse Engineering de GLPI</b>                                 | <b>4</b>  |
| 2.1      | Présentation et méthodologie . . . . .                             | 4         |
| 2.2      | Choix du périmètre élargi . . . . .                                | 4         |
| 2.3      | Table de correspondance GLPI ↔ CYnchronize . . . . .               | 4         |
| 2.4      | Simplifications structurelles majeures . . . . .                   | 6         |
| 2.5      | Adaptations Oracle . . . . .                                       | 6         |
| <b>3</b> | <b>Modèle de données</b>                                           | <b>7</b>  |
| 3.1      | Vue d'ensemble : 16 tables et 5 domaines . . . . .                 | 7         |
| 3.2      | Points-clés par domaine . . . . .                                  | 7         |
| 3.3      | Stratégie BDDR : fragmentation + réplication asymétrique . . . . . | 7         |
| 3.4      | Schéma logique (extrait) . . . . .                                 | 7         |
| 3.5      | Détail des contraintes d'intégrité métier . . . . .                | 8         |
| <b>4</b> | <b>Implémentation Oracle</b>                                       | <b>8</b>  |
| 4.1      | Tablespaces : 6 tablespaces par nœud . . . . .                     | 8         |
| 4.2      | Utilisateurs et rôles . . . . .                                    | 9         |
| 4.3      | Tables et contraintes . . . . .                                    | 10        |
| <b>5</b> | <b>Infrastructure et orchestration</b>                             | <b>10</b> |
| <b>6</b> | <b>PL/SQL et logique métier</b>                                    | <b>10</b> |
| 6.1      | Objectifs de la partie PL/SQL . . . . .                            | 10        |
| 6.2      | Package PKG_EXCEPTIONS . . . . .                                   | 11        |

|           |                                                                                   |           |
|-----------|-----------------------------------------------------------------------------------|-----------|
| 6.3       | Package PKG_FONCTIONS_METIER . . . . .                                            | 12        |
| 6.4       | Package PKG_ADMIN_PARC . . . . .                                                  | 12        |
| 6.5       | Triggers d'intégrité . . . . .                                                    | 14        |
| 6.6       | Audit automatique . . . . .                                                       | 15        |
| 6.7       | Curseur batch et statistiques . . . . .                                           | 16        |
| 6.8       | Intégration avec les autres parties . . . . .                                     | 16        |
| 6.9       | Validation . . . . .                                                              | 17        |
| <b>7</b>  | <b>Base de données répartie BDDR</b>                                              | <b>17</b> |
| 7.1       | Database links : la couche de communication inter-sites . . . . .                 | 18        |
| 7.2       | Vues matérialisées : la réplication asynchrone des référentiels . . . . .         | 18        |
| 7.3       | Vues globales distribuées : reconstruction par UNION ALL . . . . .                | 19        |
| 7.4       | Synchronisation des tables référentielles sur Pau . . . . .                       | 20        |
| <b>8</b>  | <b>Optimisation : index, cluster, plans d'exécution</b>                           | <b>20</b> |
| 8.1       | Index B-Tree : accélérer les jointures sur clés étrangères . . . . .              | 20        |
| 8.2       | Index Bitmap : compresser les filtres à faible sélectivité . . . . .              | 21        |
| 8.3       | Index composites : couvrir les combinaisons multi-critères . . . . .              | 21        |
| 8.4       | Index par fonction : recherches sur expression déterministe . . . . .             | 21        |
| 8.5       | Cluster CL_MATERIEL_ATTRIBUTION : cohabitation physique maître-détails            | 22        |
| 8.6       | Mise en cluster d'une table existante : protocole de migration . . . . .          | 22        |
| <b>9</b>  | <b>Protocole de test et résultats</b>                                             | <b>22</b> |
| 9.1       | Volumétrie et stratégie de génération des jeux de données . . . . .               | 22        |
| 9.2       | Algorithme du générateur paramétrique (peupler_tout) . . . . .                    | 24        |
| 9.3       | Protocole expérimental du benchmark . . . . .                                     | 24        |
| 9.4       | Résultats bruts du Benchmark . . . . .                                            | 25        |
| 9.5       | Analyse critique des performances et justifications des écarts . . . . .          | 26        |
| 9.6       | Validation de l'architecture distribuée : Matrice MV vs DB Link distant . . . . . | 26        |
| <b>10</b> | <b>Architecture des fichiers et exécution</b>                                     | <b>27</b> |
| 10.1      | Arborescence du dépôt . . . . .                                                   | 27        |
| 10.2      | Reproduction complète . . . . .                                                   | 27        |
| <b>11</b> | <b>Limites et axes d'amélioration</b>                                             | <b>27</b> |
| 11.1      | Limites du schéma et de l'infrastructure . . . . .                                | 27        |
| 11.2      | Limites de la logique métier . . . . .                                            | 27        |
| 11.3      | Limites BDDR et performance . . . . .                                             | 28        |
| 11.4      | Pistes d'extension . . . . .                                                      | 28        |
| <b>12</b> | <b>Conclusion</b>                                                                 | <b>28</b> |

# 1 Introduction

## 1.1 Contexte

Le projet consiste à construire une base Oracle multi-sites qui simule un système de gestion de parc IT inspiré de GLPI. L'objectif est de mettre en pratique les notions vues en cours sur un cas réaliste : tablespaces, users et rôles, contraintes d'intégrité, index et clusters, PL/SQL (packages, triggers, curseurs), bases réparties (fragmentation, réplication, vues matérialisées, vues globales), et enfin mesurer les performances avant et après optimisation.

On part d'un produit existant (GLPI) plutôt que d'inventer un modèle de toutes pièces, ce qui nous permet de faire un vrai travail de reverse engineering : on identifie les tables pertinentes, on simplifie celles qui ne servent pas, et on adapte le tout au moteur Oracle.

## 1.2 Choix du périmètre fonctionnel

Plutôt que de reproduire la totalité du modèle GLPI (plusieurs centaines de tables), nous avons retenu un **périmètre élargi** centré sur la gestion du parc informatique :

- gestion des sites et des locations (bâtiments, étages, salles),
- gestion des utilisateurs avec rôles et profils,
- gestion du parc matériel (modèles, constructeurs, attributions aux utilisateurs),
- gestion du parc réseau (équipements, VLAN, ports),
- traçabilité par journal d'audit.

Ce périmètre suffit pour démontrer la fragmentation horizontale (séparation des données opérationnelles par site Cergy/Pau) et la réplication (référentiels partagés via vues matérialisées).

## 1.3 Architecture globale

La base est distribuée sur **deux nœuds Oracle XE 21c** simulant deux sites géographiques :

| Nœud  | Conteneur Docker | Port hôte | Rôle                                |
|-------|------------------|-----------|-------------------------------------|
| Cergy | oracle-cergy     | 1521      | Maître référentiels + données Cergy |
| Pau   | oracle-pau       | 1522      | Données Pau + MV des référentiels   |

Les deux conteneurs communiquent sur un réseau Docker dédié (**oracle-net**).

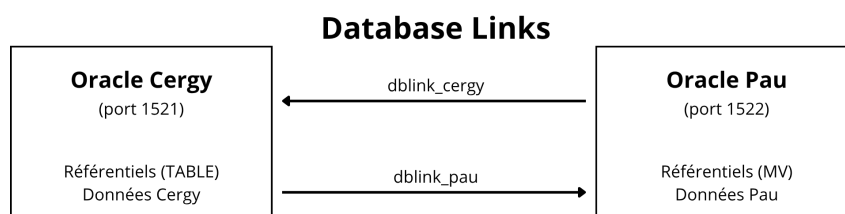


FIGURE 1 – Architecture globale et communication inter-sites

## 1.4 Stack technique

- **SGBD** : Oracle Database Express Edition 21c (image `gvenzl/oracle-xe:21-full`)
- **Orchestration** : Docker Compose
- **Langage** : SQL + PL/SQL exclusivement
- **PDB** : XEPDB1 (Pluggable Database par défaut d'Oracle XE 21c)
- **Génération de données** : 100 % PL/SQL (pas d'outil externe)

## 2 Reverse Engineering de GLPI

### 2.1 Présentation et méthodologie

GLPI (Gestionnaire Libre de Parc Informatique) est une application open source développée par Teclib' depuis 2003 sous licence GPL, aujourd'hui en version 11.0. Sa base MySQL/MariaDB compte plusieurs centaines de tables couvrant un périmètre large : inventaire, tickets ITIL, contrats, licences, machines virtuelles, etc.

Notre méthodologie : récupération du schéma officiel (`install/mysql/glpi-empty.sql` sur GitHub), identification des domaines pertinents pour notre périmètre (parc, utilisateurs, réseau, localisations), analyse des tables candidates, et décisions de simplification documentées. Le résultat : un noyau de 16 tables organisées en 5 domaines, adapté au moteur Oracle XE 21c et à une architecture distribuée.

### 2.2 Choix du périmètre élargi

Cinq domaines fonctionnels retenus : (1) Sites et localisations (référentiel partagé), (2) Utilisateurs, groupes et profils, (3) Matériels, modèles, constructeurs et historique d'attribution, (4) Équipements réseau, VLAN et ports, (5) Journal d'audit transverse. Sont volontairement exclus : tickets ITIL, problèmes, changements, validations, licences logicielles, contrats, budgets, cartouches, certificats, machines virtuelles, soit plus des deux tiers du schéma GLPI mais hors périmètre.

### 2.3 Table de correspondance GLPI ↔ CYNchronize

| Table GLPI                  | Notre table | Transformation                                                                                                                                                                                                                                                                    |
|-----------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>glpi_entities</code>  | SITE        | Hierarchie arborescente GLPI ( <code>ancestors_cache</code> , <code>completename</code> ) aplatie en deux sites plats Cergy/Pau, pivot de la fragmentation BDDR.                                                                                                                  |
| <code>glpi_locations</code> | LOCATION    | Conservation des champs essentiels ( <code>batiment</code> , <code>etage</code> , <code>salle</code> ), suppression de la hiérarchie récursive, FK explicite vers SITE.                                                                                                           |
| <code>glpi_users</code>     | UTILISATEUR | Simplification forte : 60+ colonnes GLPI (LDAP, hashes, préférences UI, photos) réduites à <code>login</code> , <code>nom</code> , <code>prenom</code> , <code>email</code> , <code>site_id</code> , <code>location_id</code> , <code>date_creation</code> , <code>actif</code> . |

| Table GLPI                                                                              | Notre table                               | Transformation                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| glpi_groups,<br>glpi_profiles                                                           | GROUPE, PROFIL                            | Conservation directe ( <b>nom</b> , <b>description</b> ). Hiérarchie de groupes GLPI aplatie. Groupe = organisationnel, Profil = applicatif.                                                                                                                                                             |
| glpi_groups_users,<br>glpi_profiles_users                                               | UTILISATEUR_GROUPE,<br>UTILISATEUR_PROFIL | Tables associatives many-to-many, PK composite. Le <b>entities_id</b> GLPI est supprimé (un utilisateur appartient à un seul site).                                                                                                                                                                      |
| glpi_computers,<br>glpi_monitors,<br>glpi_printers,<br>glpi_peripherals,<br>glpi_phones | MATERIEL                                  | Fusion majeure : GLPI dédie une table par type d'asset. Nous unifions en une seule MATERIEL avec FK vers TYPE_MATERIEL. Évite l'explosion combinatoire ( $5 \times 3$ tables référentielles). Conservation de <b>numero_serie</b> , <b>modele_id</b> , <b>statut</b> , <b>etat</b> , <b>date_achat</b> . |
| glpi_computertypes<br>(et autres)                                                       | TYPE_MATERIEL                             | Référentiel unifié de types pour tous les matériels.                                                                                                                                                                                                                                                     |
| glpi_manufacturers                                                                      | CONSTRUCTEUR                              | Conservation directe (nom uniquement).                                                                                                                                                                                                                                                                   |
| glpi_computermodels<br>(et autres)                                                      | MODELE                                    | Référentiel unifié de modèles, lié à CONSTRUCTEUR et TYPE_MATERIEL.                                                                                                                                                                                                                                      |
| users_id dans<br>glpi_computers                                                         | ATTRIBUTION                               | Table dédiée pour historiser : GLPI écrase l'utilisateur courant et perd l'historique. Notre ATTRIBUTION ( <b>date_debut</b> , <b>date_fin</b> , <b>motif</b> ) conserve l'intégralité. Cible idéale pour le cluster Oracle MATERIEL+ATTRIBUTION.                                                        |
| glpi_network-<br>equipments                                                             | EQUIPEMENT_RESEAU                         | Cœur conservé ( <b>nom</b> , type, site, location, <b>adresse_mac</b> , <b>adresse_ip</b> ), colonnes optionnelles GLPI omises.                                                                                                                                                                          |
| glpi_network-<br>equipmenttypes<br>glpi_networkports                                    | TYPE_EQUIPEMENT_<br>RESEAU<br>PORT_RESEAU | Référentiel des types réseau : switch, routeur, AP, firewall, serveur.<br>Pivot du domaine réseau : trois FK simultanées ( <b>equipement_id</b> , <b>vlan_id</b> , <b>matériel_connecte_id</b> ).                                                                                                        |
| glpi_vlans                                                                              | VLAN                                      | Conservation ( <b>numero</b> , <b>nom</b> ) + ajout d'un <b>site_id</b> : un VLAN appartient à un site donné.                                                                                                                                                                                            |
| glpi_logs                                                                               | JOURNAL_AUDIT                             | Idée du log universel conservée mais simplifiée : <b>ancien_valeur</b> et <b>nouvelle_valeur</b> en CLOB sérialisé pipe-delimited. Alimenté par triggers AFTER côté PL/SQL.                                                                                                                              |

## 2.4 Simplifications structurelles majeures

- **Unification des matériels.** GLPI applique « une table par classe d’asset » (5+ tables matériel + leurs référentiels). Nous fusionnons en MATERIEL + MODELE + CONSTRUCTEUR + TYPE\_MATERIEL (4 tables suffisantes). L’équipement réseau garde sa propre table car ses attributs (MAC, IP, ports) sont structurellement différents.
- **Aplatissement des hiérarchies.** Suppression des champs `ancestors_cache`, `sons_cache`, `level`, `completename` présents dans entities, locations, groupes GLPI. Inadapté à notre cas plat (2 sites, locations directes).
- **Ajout systématique de `site_id`.** GLPI étant mono-instance ne prévoit pas la fragmentation. Toutes nos tables opérationnelles reçoivent un `site_id` NOT NULL, qui sert de critère de partitionnement BDDR.
- **Table `ATTRIBUTION` dédiée.** Là où GLPI écrase `users_id` directement dans la table matériel (perte d’historique), nous créons une table d’historisation. Base pour les fonctions PL/SQL et cible du cluster Oracle.
- **Suppression des métadonnées GLPI.** Suppression systématique de `date_mod`, `is_deleted`, `is_template`, `is_recursive`, `pictures`, soit environ 100 colonnes en moins sur l’ensemble du schéma.

## 2.5 Adaptations Oracle

Passage MySQL/InnoDB → Oracle XE 21c, principales équivalences :

| Élément        | MySQL/GLPI                                               | Oracle/CYnchronize                                                     |
|----------------|----------------------------------------------------------|------------------------------------------------------------------------|
| Auto-incrément | <code>int unsigned</code><br><code>AUTO_INCREMENT</code> | <code>NUMBER GENERATED ALWAYS AS IDENTITY</code>                       |
| Booléen        | <code>tinyint</code>                                     | <code>NUMBER(1) + CHECK IN (0,1)</code>                                |
| Texte long     | <code>longtext</code>                                    | <code>CLOB</code>                                                      |
| Stockage       | InnoDB transparent                                       | Tablespaces explicites, <code>AUTOEXTEND</code> , <code>MAXSIZE</code> |
| FK             | Souvent omises dans GLPI                                 | Systématiques, nommées                                                 |
| Polymorphisme  | <code>itemtype + items_id</code>                         | Évité : FK fortes par domaine                                          |

## 3 Modèle de données

### 3.1 Vue d'ensemble : 16 tables et 5 domaines

| Domaine              | Tables                                                              | Nature BDDR                           |
|----------------------|---------------------------------------------------------------------|---------------------------------------|
| Sites & localisation | SITE, LOCATION                                                      | Référentiel partagé (maître Cergy)    |
| Utilisateurs         | UTILISATEUR, GROUPE, PROFIL, UTILISATEUR_GROUPE, UTILISATEUR_PROFIL | Fragmenté (UTILISATEUR) + référentiel |
| Matériels            | MATERIEL, ATTRIBUTION, TYPE_MATERIEL, CONSTRUCTEUR, MODELE          | Fragmenté + référentiel               |
| Réseau               | EQUIPEMENT_RESEAU, VLAN, PORT_RESEAU, TYPE_EQUIPEMENT_RESEAU        | Fragmenté + référentiel               |
| Audit                | JOURNAL_AUDIT                                                       | Transverse, local                     |

### 3.2 Points-clés par domaine

- **Sites/localisation** : SITE est le pivot du modèle (critère de fragmentation). LOCATION précise bâtiment/étage/salle au sein d'un site.
- **Utilisateurs** : Découpage volontaire groupe (organisationnel) / profil (applicatif). Un utilisateur peut être dans plusieurs groupes mais a un seul profil.
- **Matériels** : Domaine au volume le plus fort. Colonne `numero_serie` unique (clé fonctionnelle). Colonnes `statut` et `etat` à basse cardinalité, cibles d'index bitmap. `ATTRIBUTION` active = `date_fin` IS NULL.
- **Réseau** : `PORT_RESEAU` est le pivot (3 FK). Unicité MAC/IP gérée par triggers PL/SQL pour gérer les cas particuliers (NULL, 0.0.0.0).
- **Audit** : Journal append-only, CLOB sérialisé pipe-delimited. Alimenté par triggers AFTER sur 4 tables sensibles.

### 3.3 Stratégie BDDR : fragmentation + réplication asymétrique

- **Fragmentation horizontale** sur les tables opérationnelles via `site_id`. Chaque nœud ne contient que ses propres lignes.
- **Réplication asymétrique** des 8 tables référentielles : Cergy est maître, Pau reçoit des vues matérialisées en lecture seule via dblink.

Justification métier : un nouveau modèle ou type est rare et validé centralement ; les opérations sur les matériels/utilisateurs/équipements sont quotidiennes et locales.

### 3.4 Schéma logique (extrait)

```
SITE (id, code, nom, adresse)
LOCATION (id, site_id -> SITE, batiment, etage, salle)

UTILISATEUR (id, login, nom, prenom, email, site_id -> SITE,
             location_id -> LOCATION, date_creation, actif)
GROUPE (id, nom, description)
PROFIL (id, nom, description)
UTILISATEUR_GROUPE (utilisateur_id, groupe_id)
UTILISATEUR_PROFIL (utilisateur_id, profil_id)
```

```

TYPE_MATERIEL (id, libelle)
CONSTRUCTEUR (id, nom)
MODELE (id, constructeur_id -> CONSTRUCTEUR,
        type_materiel_id -> TYPE_MATERIEL, reference)
MATERIEL (id, numero_serie, modele_id -> MODELE,
        site_id -> SITE, location_id -> LOCATION,
        date_achat, statut, etat)
ATTRIBUTION (id, materiel_id -> MATERIEL,
        utilisateur_id -> UTILISATEUR,
        date_debut, date_fin, motif)

TYPE_EQUIPEMENT_RESEAU (id, libelle)
EQUIPEMENT_RESEAU (id, nom, type_id -> TYPE_EQUIPEMENT_RESEAU,
        site_id -> SITE, location_id -> LOCATION,
        adresse_mac, adresse_ip)
VLAN (id, numero, nom, site_id -> SITE)
PORT_RESEAU (id, equipement_id -> EQUIPEMENT_RESEAU, numero,
        vlan_id -> VLAN,
        materiel_connecte_id -> MATERIEL, statut)

JOURNAL_AUDIT (id, table_concernee, operation, id_enregistrement,
        ancien_valeur CLOB, nouvelle_valeur CLOB,
        utilisateur_oracle, date_action)

```

### 3.5 Détail des contraintes d'intégrité métier

Au-delà des 21 clés étrangères nommées, le schéma intègre cinq contraintes CHECK qui encodent les énumérations métier et trois contraintes d'unicité fonctionnelles :

- CHK\_MATERIEL\_STATUT : statut  $\in$  {en\_service, en\_stock, en\_reparation, reforme, pret}
- CHK\_MATERIEL\_ETAT : etat  $\in$  {fonctionnel, defectueux, obsolete, neuf}
- CHK\_PORT\_STATUT : statut  $\in$  {libre, utilise, desactive}
- CHK\_UTILISATEUR\_ACTIF : actif  $\in$  {0, 1}
- CHK\_ATTRIBUTION\_DATES : date\_fin IS NULL OR date\_fin  $\geq$  date\_debut
- UK\_SITE\_CODE, UK\_UTILISATEUR\_LOGIN, UK\_MATERIEL\_SERIE : trois unicités fonctionnelles.

## 4 Implémentation Oracle

### 4.1 Tablespaces : 6 tablespaces par nœud

Trois raisons à ce découpage : (1) séparation I/O physique (datafiles distincts pouvant aller sur des disques différents), (2) profils de charge hétérogènes (référentiel lecture pure / matériels lecture-écriture / audit append-only), (3) séparation données/index permettant le parallélisme physique des lectures.



| Tablespace       | Contenu                                       | Profil I/O                    | Taille init. |
|------------------|-----------------------------------------------|-------------------------------|--------------|
| TBS_REFERENTIEL  | Sites, profils, types, constructeurs, modèles | Lecture, petit volume         | 50 Mo        |
| TBS_UTILISATEURS | Utilisateurs, associations                    | Lecture/écriture modérée      | 50 Mo        |
| TBS_MATERIELS    | Matériels, attributions (volume principal)    | Lecture/écriture forte        | 100 Mo       |
| TBS_RESEAU       | Équipements, VLANs, ports                     | Lecture, faible volume        | 50 Mo        |
| TBS_AUDIT        | Journal d'audit                               | Append-only, forte croissance | 100 Mo       |
| TBS_INDEX        | Tous les index                                | Lecture/écriture index        | 100 Mo       |

Tous AUTOEXTEND ON avec MAXSIZE 500 Mo ou 1 Go selon profil. Exemple :

```
CREATE TABLESPACE TBS_MATERIELS DATAFILE 'tbs_materiels.dbf'
SIZE 100M AUTOEXTEND ON NEXT 20M MAXSIZE 1G;
```

## 4.2 Utilisateurs et rôles

Principe RBAC strict : aucun privilège n'est accordé directement à un utilisateur, tout transite par des rôles. Trois avantages : maintenabilité (modifier le rôle, pas chaque compte), cohérence (mêmes droits par construction pour un même métier), auditabilité (matrice rôles × privilèges documentable).

**Compte propriétaire GLPI\_OWNER** : propriétaire de tous les objets, quotas illimités sur les 6 tablespaces, aucun applicatif ne s'y connecte directement.

Cinq rôles métier + un compte de réplication :

| Rôle               | Métier          | Privilèges                                                                        |
|--------------------|-----------------|-----------------------------------------------------------------------------------|
| ROLE_ADMIN_PARC    | Admin parc IT   | CRUD matériels/attributions/équipements ; lecture référentiels ; EXECUTE packages |
| ROLE_TECH_RESEAU   | Tech réseau     | CRUD réseau ; lecture matériels/utilisateurs                                      |
| ROLE_GESTION_USERS | Gestion comptes | CRUD utilisateurs/groupes/profils ; lecture matériels                             |
| ROLE_CONSULT       | Reporting       | SELECT sur vues globales uniquement                                               |
| ROLE_AUDITEUR      | Auditeur        | SELECT sur JOURNAL_AUDIT                                                          |

**Compte de réplication link\_user** : SELECT strictement limité sur les tables nécessaires aux dblinks. Cloisonnement essentiel : un compte applicatif aurait un risque d'exploitation incomparable.

Cinq comptes de démonstration créés : **admin\_parc**, **tech\_reseau**, **gestion\_users**, **consultant**, **auditeur**, chacun rattaché à son rôle, avec uniquement CREATE SESSION en plus.

```
CREATE ROLE ROLE_ADMIN_PARC;
GRANT SELECT, INSERT, UPDATE, DELETE ON GLPI_OWNER.MATERIEL
TO ROLE_ADMIN_PARC;
CREATE USER admin_parc IDENTIFIED BY admin123
DEFAULT TABLESPACE TBS_UTILISATEURS;
GRANT ROLE_ADMIN_PARC, CREATE SESSION TO admin_parc;
```

### 4.3 Tables et contraintes

Chaque table créée avec PK GENERATED ALWAYS AS IDENTITY et tablespace explicite. DDL et contraintes séparés en deux scripts pour faciliter la maintenance. Exemple :

```
CREATE TABLE MATERIEL (  
  id                NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  numero_serie     VARCHAR2(100) NOT NULL,  
  modele_id        NUMBER          NOT NULL,  
  site_id          NUMBER          NOT NULL,  
  location_id      NUMBER,  
  date_achat       DATE,  
  statut           VARCHAR2(30) DEFAULT 'en_service' NOT NULL,  
  etat             VARCHAR2(30) DEFAULT 'fonctionnel' NOT NULL  
) TABLESPACE TBS_MATERIELS;
```

Bilan des contraintes : 21 clés étrangères (toutes nommées), 5 contraintes CHECK (énumérations statut/etat/booléens, cohérence dates), 3 contraintes UNIQUE (SITE.code, UTILISATEUR.login, MATERIEL.numero\_serie). Les règles inter-tables plus complexes (cohérence site matériel/utilisateur, unicité conditionnelle MAC/IP) sont déléguées aux triggers PL/SQL.

## 5 Infrastructure et orchestration

L'environnement repose sur deux conteneurs Oracle XE 21c (gvenz1/oracle-xe:21-full) orchestrés par Docker Compose. Cergy expose 1521, Pau expose 1522, les deux conteneurs partagent un réseau bridge `oracle-net` permettant la résolution DNS interne (essentielle pour les dblinks de la couche BDDR).

Un script `init.sh` à la racine rejoue tout le pipeline dans l'ordre des dépendances : Docker → tablespaces → tables → contraintes → users/rôles → MV logs Cergy → PL/SQL → dblinks et MV → seed data → table de benchmark. Cet ordre encode les dépendances inter-membres et garantit la reproductibilité complète en une seule commande.

## 6 PL/SQL et logique métier

Cette partie porte sur la **logique métier PL/SQL**. Elle se situe au-dessus du schéma relationnel sous-jacent et sert à garantir que les opérations métier restent cohérentes, auditables et réutilisables. Elle comprend les packages, les fonctions, les procédures, les triggers d'intégrité, les triggers d'audit et un traitement batch avec curseur explicite.

Cette partie dépend du schéma défini précédemment, car les packages et triggers ont besoin des tables MATERIEL, ATTRIBUTION, UTILISATEUR, EQUIPEMENT\_RESEAU, VLAN, PORT\_RESEAU et JOURNAL\_AUDIT. Elle a également une interdépendance avec la couche BDDR, car la procédure de transfert inter-sites utilise les database links `dblink_cergy` et `dblink_pau`.

### 6.1 Objectifs de la partie PL/SQL

La partie PL/SQL répond à quatre besoins principaux :

- centraliser les erreurs métier avec des codes Oracle explicites ;
- fournir des fonctions et procédures réutilisables pour les opérations courantes du parc ;
- garantir certaines règles métier directement dans la base par triggers ;
- tracer automatiquement les changements sensibles dans un journal d'audit.

Les fichiers livrés sont les suivants :

| Fichier                                | Rôle                                                              |
|----------------------------------------|-------------------------------------------------------------------|
| plsql/01_exceptions.sql                | Package PKG_EXCEPTIONS, codes d'erreur métier -20001 à -20005     |
| plsql/02_pkg_fonctions_metier_spec.sql | Spécification du package PKG_FONCTIONS_METIER                     |
| plsql/03_pkg_fonctions_metier_body.sql | Corps des fonctions métier                                        |
| plsql/04_pkg_admin_parc_spec.sql       | Spécification du package PKG_ADMIN_PARC                           |
| plsql/05_pkg_admin_parc_body.sql       | Corps des procédures métier                                       |
| plsql/06_triggers_integrite.sql        | Triggers d'intégrité                                              |
| plsql/07_triggers_audit.sql            | Procédure d'audit, fonctions de sérialisation et triggers d'audit |
| plsql/08_curseur_batch.sql             | Table STATISTIQUES_PARC et procédure batch avec curseur explicite |

Le choix a été de regrouper la logique stable dans des packages, et de réserver les triggers aux règles qui doivent être appliquées quelle que soit l'origine de l'opération : script SQL, procédure, utilisateur applicatif ou insertion massive de test.

## 6.2 Package PKG\_EXCEPTIONS

Le premier script crée le package PKG\_EXCEPTIONS. Son rôle est de centraliser les exceptions métier et d'éviter d'éparpiller des appels directs à RAISE\_APPLICATION\_ERROR dans tous les fichiers PL/SQL.

Les codes utilisés sont :

| Code   | Exception                | Signification                                              |
|--------|--------------------------|------------------------------------------------------------|
| -20001 | x_site_incompatible      | Deux objets manipulés appartiennent à des sites différents |
| -20002 | x_materiel_deja_attribue | Le matériel possède déjà une attribution active            |
| -20003 | x_mac_dupliee            | Une adresse MAC est déjà utilisée                          |
| -20004 | x_ip_dupliee             | Une adresse IP est déjà utilisée                           |
| -20005 | x_attribution_invalide   | Attribution, matériel ou utilisateur invalide              |

Chaque exception est déclarée dans la spécification du package et associée à son code grâce à PRAGMA EXCEPTION\_INIT. Le corps du package contient une procédure interne raise\_error, puis une procédure publique par type d'erreur :

```
PROCEDURE raise_site_incompatible(p_message IN VARCHAR2 DEFAULT NULL);
PROCEDURE raise_materiel_deja_attribue(p_message IN VARCHAR2 DEFAULT NULL);
PROCEDURE raise_mac_dupliee(p_message IN VARCHAR2 DEFAULT NULL);
PROCEDURE raise_ip_dupliee(p_message IN VARCHAR2 DEFAULT NULL);
PROCEDURE raise_attribution_invalide(p_message IN VARCHAR2 DEFAULT NULL);
```

L'intérêt principal est la maintenabilité. Si un message ou un code doit être modifié, il suffit de le changer à un seul endroit. Le package joue aussi un rôle documentaire : il donne une vue immédiate des erreurs métier que l'application peut produire.

## 6.3 Package PKG\_FONCTIONS\_METIER

Le package PKG\_FONCTIONS\_METIER regroupe des fonctions de lecture utilisées par les procédures, les requêtes d’inventaire et le traitement statistique.

### 6.3.1 Âge d’un matériel

La fonction `age_materiel_mois(p_materiel_id)` lit la date d’achat du matériel et calcule son âge avec `MONTHS_BETWEEN(SYSDATE, date_achat)`. Le résultat est tronqué en nombre de mois entiers.

Si le matériel n’existe pas, la fonction lève une erreur métier via `PKG_EXCEPTIONS.raise_attribution_invalide`.

### 6.3.2 Obsolescence

La fonction `est_obsolete(p_materiel_id)` retourne 1 si le matériel a au moins 60 mois, sinon 0.

Cette fonction retourne volontairement un `NUMBER(0/1)`, car Oracle ne permet pas d’utiliser directement un `BOOLEAN` PL/SQL dans un `SELECT`. Avec un retour numérique, la fonction est utilisable dans :

- une projection SQL ;
- un `GROUP BY` ;
- un curseur ;
- une vue ou un reporting.

Ce choix permet par exemple au curseur batch de regrouper les matériels selon leur obsolescence.

### 6.3.3 Nombre de matériels attribués

La fonction `nb_materiels_attribues(p_utilisateur_id)` compte les attributions actives d’un utilisateur :

```
WHERE utilisateur_id = p_utilisateur_id
AND date_fin IS NULL
```

La règle retenue est simple : une attribution active est une ligne de `ATTRIBUTION` dont `date_fin` est nulle.

### 6.3.4 Taux d’occupation des ports

La fonction `taux_occupation_ports(p_equipement_id)` calcule le pourcentage de ports utilisés sur un équipement réseau. Un port est considéré comme utilisé si :

- son statut vaut `'utilise'`, ou
- `materiel_connecte_id` n’est pas nul.

La fonction retourne 0 si l’équipement ne possède aucun port, ce qui évite une division par zéro.

## 6.4 Package PKG\_ADMIN\_PARC

Le package PKG\_ADMIN\_PARC regroupe les procédures métier principales. L’objectif est d’éviter que les opérations sensibles soient faites par des scripts SQL dispersés. Le package impose une séquence d’actions contrôlée et vérifie les règles métier avant de modifier les tables.

### 6.4.1 Attribution d'un matériel

La procédure `attribuer_materiel(p_materiel_id, p_utilisateur_id, p_motif)` réalise l'attribution d'un matériel à un utilisateur.

Elle commence par vérifier que le matériel et l'utilisateur appartiennent au même site. Cette vérification utilise les colonnes `site_id` de `MATERIEL` et `UTILISATEUR`. Si les sites sont différents, l'erreur `x_site_incompatible` est levée.

Ensuite, la procédure vérifie qu'il n'existe pas déjà une attribution active pour ce matériel :

```
SELECT COUNT(*)
FROM ATTRIBUTION
WHERE materiel_id = p_materiel_id
AND date_fin IS NULL;
```

Si le matériel est libre, la procédure insère une ligne dans `ATTRIBUTION` avec `date_debut = SYSDATE`, puis met le statut du matériel à 'pret'.

Cette procédure correspond à une opération métier classique d'un outil de gestion de parc : associer un équipement à un utilisateur tout en gardant l'historique dans une table dédiée.

### 6.4.2 Clôture d'une attribution

La procédure `cloturer_attribution(p_attribution_id, p_motif)` clôture une attribution active.

Elle lit d'abord la ligne avec `FOR UPDATE` :

```
SELECT date_fin
INTO v_date_fin
FROM ATTRIBUTION
WHERE id = p_attribution_id
FOR UPDATE;
```

Le `FOR UPDATE` verrouille la ligne pendant la transaction. Cela évite qu'une autre session clôture simultanément la même attribution.

Si `date_fin` est déjà renseignée, la procédure refuse l'opération. Sinon, elle met `date_fin = SYSDATE` et conserve ou remplace le motif selon le paramètre reçu.

### 6.4.3 Transfert inter-sites

`transférer_materiel_inter_sites(p_materiel_id, p_site_dest_id, p_location_dest_id)` est la procédure la plus liée à la BDDR.

Son déroulement est le suivant :

1. Lecture du matériel local avec `SELECT * INTO ... FOR UPDATE`.
2. Vérification que le site de destination est différent du site courant.
3. Vérification optionnelle que la location de destination appartient bien au site de destination.
4. Clôture de toutes les attributions actives du matériel.
5. Détermination du code du site destination (`CERGY` ou `PAU`).
6. Insertion distante dans `MATERIEL@dblink_cergy` ou `MATERIEL@dblink_pau`.
7. Suppression locale des attributions puis du matériel.

L'insertion distante est faite avec `EXECUTE IMMEDIATE`. Ce choix permet au package de compiler

même si les database links ne sont pas encore présents au moment de la compilation. L'appel réel, lui, nécessite évidemment que les database links aient été créés.

Extrait logique :

```
IF v_site_dest_code = 'CERGY' THEN
    EXECUTE IMMEDIATE 'INSERT INTO MATERIEL@dblink_cergy (...) VALUES (...)'
    USING ...;
ELSIF v_site_dest_code = 'PAU' THEN
    EXECUTE IMMEDIATE 'INSERT INTO MATERIEL@dblink_pau (...) VALUES (...)'
    USING ...;
END IF;
```

Cette procédure illustre l'interdépendance entre la logique métier et la BDDR : la logique métier est dans le package PL/SQL, mais son exécution distribuée dépend de l'infrastructure BDDR.

#### 6.4.4 Inventaire d'un site

La procédure `generer_inventaire_site(p_site_id, p_cursor OUT SYS_REFCURSOR)` ouvre un curseur vers l'inventaire détaillé d'un site.

La requête joint : MATERIEL, MODELE, CONSTRUCTEUR, TYPE\_MATERIEL, LOCATION.

Elle ajoute aussi les résultats de `age_materiel_mois` et `est_obsolete` dans la projection.

Le choix d'un SYS\_REFCURSOR permet de retourner un ensemble de lignes à l'appelant sans charger toute la réponse dans une collection PL/SQL. L'appelant peut ensuite parcourir le curseur à son rythme.

### 6.5 Triggers d'intégrité

Le fichier `06_triggers_integrite.sql` crée quatre triggers chargés de garantir les règles métier qui ne peuvent pas être exprimées simplement par des contraintes CHECK ou FOREIGN KEY.

#### 6.5.1 Cohérence site matériel / utilisateur

Le trigger `trg_materiel_cohesion_site` s'exécute avant insertion ou modification sur `ATTRIBUTION`. Il compare `MATERIEL.site_id` et `UTILISATEUR.site_id`.

Si les deux valeurs sont différentes, l'attribution est refusée.

Cette règle est importante dans une base fragmentée par site : un matériel de Cergy ne doit pas être attribué directement à un utilisateur de Pau.

#### 6.5.2 Cohérence port / VLAN

Le trigger `trg_port_vlan_meme_site` s'exécute sur `PORT_RESEAU`. Si un port est associé à un VLAN, le trigger vérifie que l'équipement réseau du port et le VLAN appartiennent au même site.

Cela évite une incohérence réseau : un port physique situé sur un équipement de Cergy ne doit pas être rattaché à un VLAN déclaré à Pau.

#### 6.5.3 Unicité conditionnelle MAC et IP

Les triggers `trg_unique_mac` et `trg_unique_ip` garantissent l'unicité des adresses MAC et IP sur `EQUIPEMENT_RESEAU`.

Ces règles ne sont pas de simples contraintes UNIQUE, car certains cas doivent rester autorisés :

- `adresse_mac` peut être NULL ;
- `adresse_ip` peut être NULL ;
- `adresse_ip = '0.0.0.0'` est acceptée comme valeur réservée.

Les deux triggers sont implémentés en **compound triggers** pour éviter l'erreur Oracle **ORA-04091 : table is mutating**.

Le principe est le suivant :

- **BEFORE EACH ROW** collecte les nouvelles valeurs dans un tableau PL/SQL ;
- **AFTER STATEMENT** relit la table et vérifie que chaque valeur collectée n'apparaît pas plus d'une fois.

Ce choix est plus robuste qu'un trigger ligne classique, car un trigger **FOR EACH ROW** qui relit sa propre table peut échouer dès qu'il traite plusieurs lignes.

## 6.6 Audit automatique

Le fichier `07_triggers_audit.sql` met en place une couche d'audit automatique. Le but est de tracer les changements importants sans dépendre du code applicatif.

### 6.6.1 Procédure commune d'audit

La procédure `ecrire_audit` insère une ligne dans `JOURNAL_AUDIT` avec :

- le nom de la table concernée ;
- l'opération (`INSERT`, `UPDATE`, `DELETE`) ;
- l'identifiant de l'enregistrement ;
- l'ancienne valeur sérialisée ;
- la nouvelle valeur sérialisée ;
- l'utilisateur Oracle ;
- la date de l'action.

Le journal est donc alimenté par la base elle-même, indépendamment de la procédure ou de l'utilisateur qui a déclenché l'opération.

### 6.6.2 Fonctions de sérialisation

Quatre fonctions de sérialisation convertissent les lignes en texte stockable dans un `CLOB` :

| Fonction                            | Table couverte                 |
|-------------------------------------|--------------------------------|
| <code>serializer_materiel</code>    | <code>MATERIEL</code>          |
| <code>serializer_utilisateur</code> | <code>UTILISATEUR</code>       |
| <code>serializer_attribution</code> | <code>ATTRIBUTION</code>       |
| <code>serializer_equipement</code>  | <code>EQUIPEMENT_RESEAU</code> |

Le format est volontairement simple : une suite de couples `champ=valeur`, séparés par des points-virgules. Ce format est suffisant pour le projet et lisible lors d'une démonstration.

### 6.6.3 Triggers d'audit

Quatre triggers **AFTER INSERT OR UPDATE OR DELETE** appellent `ecrire_audit` :

| Trigger                     | Table auditée     |
|-----------------------------|-------------------|
| trg_audit_materiel          | MATERIEL          |
| trg_audit_utilisateur       | UTILISATEUR       |
| trg_audit_attribution       | ATTRIBUTION       |
| trg_audit_equipement_reseau | EQUIPEMENT_RESEAU |

Le choix de triggers **AFTER** est volontaire. On veut tracer uniquement les opérations qui ont réussi. Si l'audit était fait en **BEFORE**, il pourrait enregistrer une action ensuite annulée par une contrainte ou un autre trigger.

Pour un **INSERT**, seule la nouvelle valeur est stockée. Pour un **DELETE**, seule l'ancienne valeur est stockée. Pour un **UPDATE**, les deux versions sont enregistrées, ce qui permet de comprendre précisément ce qui a changé.

## 6.7 Curseur batch et statistiques

Le fichier `08_curseur_batch.sql` crée la table `STATISTIQUES_PARC` et la procédure `recalculer_statistiques_parc`.

La table contient : `site_id`, `type_materiel`, `etat`, `est_obsolete`, `nombre`, `date_calcul`.

La procédure commence par vider les anciennes statistiques, puis parcourt un curseur explicite :

```
CURSOR c_site_materiel IS
  SELECT m.site_id,
         tm.libelle AS type_materiel,
         m.etat,
         PKG_FONCTIONS_METIER.est_obsolete(m.id) AS est_obsolete,
         COUNT(*) AS nombre
  FROM MATERIEL m
  JOIN MODELE mo ON mo.id = m.modele_id
  JOIN TYPE_MATERIEL tm ON tm.id = mo.type_materiel_id
  GROUP BY m.site_id,
           tm.libelle,
           m.etat,
           PKG_FONCTIONS_METIER.est_obsolete(m.id);
```

Chaque ligne du curseur est insérée dans `STATISTIQUES_PARC`.

Cette partie répond explicitement à la notion de **curseur PL/SQL** demandée dans le sujet. Une solution en `INSERT INTO ... SELECT ...` aurait été plus courte, mais le curseur explicite montre clairement le traitement ligne par ligne attendu dans le cadre pédagogique.

## 6.8 Intégration avec les autres parties

Cette partie s'intègre directement avec les trois autres responsabilités :

- avec les fondations du schéma, car elle repose sur les tables, contraintes et rôles du schéma ;
- avec la couche BDDR, car `transférer_materiel_inter_sites` utilise les database links et parce que les fonctions PL/SQL apparaissent dans les requêtes d'inventaire et de statistiques ;
- avec les jeux de tests, car les triggers d'audit et d'intégrité sont actifs pendant les insertions de jeux de données.

Un point important a été de garder les scripts compatibles avec l'ordre d'initialisation global :

```
tablespaces -> owner -> tables -> constraints -> users/roles -> MV logs ->
```



```
PL/SQL -> BDDR -> seed data -> benchmark
```

Les packages sont compilés avant les tests et avant les scénarios de benchmark. Les triggers d'intégrité peuvent donc bloquer les données incohérentes, et les triggers d'audit peuvent tracer les insertions générées par les jeux de test.

La procédure de transfert inter-sites a été écrite avec `EXECUTE IMMEDIATE` afin que le package compile même si les db links ne sont pas encore créés. Cela a facilité le travail en parallèle des branches.

## 6.9 Validation

La compilation a été validée localement sur le nœud `oracle-cergy` avec Oracle XE 21c et le PDB XEPDB1.

Les scripts PL/SQL ont été exécutés dans l'ordre suivant :

```
01_exceptions.sql
02_pkg_fonctions_metier_spec.sql
03_pkg_fonctions_metier_body.sql
04_pkg_admin_parc_spec.sql
05_pkg_admin_parc_body.sql
06_triggers_integrite.sql
07_triggers_audit.sql
08_curseur_batch.sql
```

Après compilation, la requête suivante ne retournait aucun objet invalide :

```
SELECT object_type, object_name, status
FROM user_objects
WHERE status <> 'VALID'
ORDER BY object_type, object_name;
```

Résultat :

```
no rows selected
```

Cela confirme que les packages, triggers, fonctions, procédures et la table statistique compilent correctement ensemble sur le schéma `GLPI_OWNER`.

La validation complète du transfert inter-sites dépend de l'exécution des deux nœuds Oracle et de la création des database links. En revanche, la compilation du package ne dépend pas des db links grâce à l'utilisation de SQL dynamique.

## 7 Base de données répartie BDDR

La partie BDDR de CYnchronize repose sur trois mécanismes : les **database links** pour communiquer entre les nœuds, les **vues matérialisées** pour répliquer les référentiels partagés, et les **vues globales** en `UNION ALL` pour reconsolider les fragments. Ensemble, ces trois éléments permettent à chaque nœud de garder ses données opérationnelles locales tout en accédant à une vue globale du parc.

## 7.1 Database links : la couche de communication inter-sites

La communication entre les deux nœuds passe par deux **database links** unidirectionnels, créés dans le schéma **GLPI\_OWNER** de chaque instance. Chaque lien définit un compte distant et une chaîne de connexion : à chaque requête sur une table préfixée par **@dblink\_xxx**, le moteur ouvre une session TCP/1521 vers la base distante.

```
-- Sur Cergy : pointe vers Pau
CREATE DATABASE LINK dblink_pau
  CONNECT TO link_user IDENTIFIED BY link123
  USING '(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP) (HOST=oracle-pau) (PORT=1521))
        (CONNECT_DATA=(SERVICE_NAME=XEPDB1)))';

-- Sur Pau : pointe vers Cergy
CREATE DATABASE LINK dblink_cergy
  CONNECT TO link_user IDENTIFIED BY link123
  USING '(DESCRIPTION=(ADDRESS=(PROTOCOL=TCP) (HOST=oracle-cergy) (PORT=1521))
        (CONNECT_DATA=(SERVICE_NAME=XEPDB1)))';
```

Deux choix méritent d'être expliqués :

1. **Chaîne TNS complète plutôt qu'un alias.** La clause **USING** reçoit la description TNS entière ((**DESCRIPTION**=(**ADDRESS**=...)...)) au lieu d'un simple nom de service. Les conteneurs Docker n'ont pas de fichier **tnsnames.ora** partagé : on doit donc passer les paramètres en clair. Le nom d'hôte **oracle-pau** est résolu par le DNS interne du réseau Docker **oracle-net**.
2. **Utilisateur dédié link\_user.** Plutôt que de connecter les db links avec **GLPI\_OWNER** (qui possède tous les droits), on crée un compte séparé **link\_user** avec uniquement **CREATE SESSION** et quelques **SELECT** sur les tables nécessaires. Si jamais la chaîne TNS fuit, l'attaquant n'a accès qu'en lecture à un sous-ensemble du schéma, sans pouvoir écrire ni exécuter du PL/SQL.

## 7.2 Vues matérialisées : la réplication asynchrone des référentiels

Les huit tables référentielles maîtres sur Cergy (**SITE**, **LOCATION**, **TYPE\_MATERIEL**, **CONSTRUCTEUR**, **MODELE**, **TYPE\_EQUIPEMENT\_RESEAU**, **GROUPE**, **PROFIL**) sont **répliquées sur Pau** par autant de **vues matérialisées** (**MV\_\***) :

```
CREATE MATERIALIZED VIEW MV_SITE
  BUILD IMMEDIATE
  REFRESH FAST ON DEMAND
  WITH PRIMARY KEY
  AS SELECT * FROM GLPI_OWNER.SITE@dblink_cergy;
-- ... idem pour LOCATION, TYPE_MATERIEL, CONSTRUCTEUR, MODELE,
-- TYPE_EQUIPEMENT_RESEAU, GROUPE, PROFIL.
```

Chaque clause a son rôle :

- **BUILD IMMEDIATE** : la MV est peuplée dès sa création, donc aucune lecture ultérieure ne refait l'aller-retour vers Cergy.
- **REFRESH FAST** : seules les lignes modifiées depuis le dernier refresh sont synchronisées (par opposition à **REFRESH COMPLETE** qui relit toute la table source). Pour que ça fonctionne, il faut créer des **MV logs** sur les tables maîtres côté Cergy (fichier **perf/00\_mv\_logs\_cergy.sql**).
- **ON DEMAND** : le rafraîchissement ne se fait pas automatiquement à chaque modification, mais quand on l'appelle. C'est volontaire : ça découple les écritures sur Cergy de la

disponibilité du réseau vers Pau.

- **WITH PRIMARY KEY** : la clé primaire de la table source sert d'identifiant pour les lignes répliquées, ce qui simplifie le delta lors du FAST refresh.

### 7.2.1 Rafraîchissement automatique via DBMS\_SCHEDULER

Pour que les référentiels sur Pau restent à jour sans intervention manuelle, on a posé un job DBMS\_SCHEDULER :

```
DBMS_SCHEDULER.CREATE_JOB(  
  job_name      => 'JOB_REFRESH_REFERENTIEL',  
  job_type      => 'PLSQL_BLOCK',  
  job_action    => 'BEGIN DBMS_MVIEW.REFRESH(''MV_SITE'', ''C''); ... END;',  
  repeat_interval => 'FREQ=HOURLY',  
  enabled       => TRUE  
);
```

Ce job rafraîchit les huit MV chaque heure. Petit piège qui nous a fait perdre du temps : il faut le privilège CREATE JOB sur GLPI\_OWNER, sinon DBMS\_SCHEDULER.CREATE\_JOB échoue avec ORA-27486 insufficient privileges. On l'a ajouté dans schema/04\_owner\_\*.sql. Le job peut être inspecté ou lancé manuellement :

```
SELECT job_name, state, last_run_date, next_run_date FROM user_scheduler_jobs;  
EXEC DBMS_SCHEDULER.RUN_JOB('JOB_REFRESH_REFERENTIEL');
```

### 7.3 Vues globales distribuées : reconstruction par UNION ALL

Pour reconstituer la **table logique globale** par-dessus la fragmentation horizontale, trois vues consolidantes ont été créées sur chaque nœud avec UNION ALL :

```
CREATE OR REPLACE VIEW V_MATERIELS_GLOBAL AS  
  SELECT id, numero_serie, ..., 'CERGY' AS source FROM MATERIEL  
  UNION ALL  
  SELECT id, numero_serie, ..., 'PAU' AS source  
  FROM GLPI_OWNER.MATERIEL@dblink_pau;
```

Les trois vues globales sont V\_MATERIELS\_GLOBAL, V\_UTILISATEURS\_GLOBAL et V\_EQUIPEMENTS\_RESEAU\_GLOBAL. Trois points à expliquer :

1. **UNION ALL et pas UNION.** Les fragments sont disjoints par construction : la clé `site_id` répartit chaque ligne sur un seul nœud, donc il n'y a aucun doublon à dédupliquer. UNION ferait un tri puis un hachage pour rien ; UNION ALL empile simplement les deux fragments, c'est plus rapide.
2. **Colonne source ajoutée.** Une colonne 'CERGY' ou 'PAU' est concaténée à chaque branche. Ça permet de savoir d'où vient une ligne sans avoir à interroger les db links un par un. C'est notamment utile pour les agrégations par site (Q6 du benchmark).
3. **Préfixe GLPI\_OWNER. obligatoire.** Toute table appelée via db link doit être qualifiée par son schéma (GLPI\_OWNER.MATERIEL@dblink\_pau). En effet, le lien se connecte avec `link_user` qui ne possède pas les tables et n'a pas de synonyme public dessus.

Les vues sont déclarées en CREATE OR REPLACE pour pouvoir les recompiler à chaud, notamment après la mise en cluster de MATERIEL (section 8). Elles sont accordées au rôle ROLE\_CONSULT : l'utilisateur de démo `consultant` peut les lire sans avoir accès direct aux tables fragmentées.

## 7.4 Synchronisation des tables référentielles sur Pau

Petit détail d'implémentation qui mérite d'être expliqué : sur Pau, on a en parallèle des **tables vides** (créées par le script de schéma symétrique entre les deux nœuds) et des **vues matérialisées MV\_\* peuplées via db link**. Cette duplication n'est pas un choix esthétique mais une contrainte Oracle : les **clés étrangères** des tables opérationnelles fragmentées (MATERIEL, UTILISATEUR, VLAN, EQUIPEMENT\_RESEAU) ne peuvent référencer qu'une **table physique**, pas une vue matérialisée.

Pour résoudre ça, le script `tests/02_referentiels_pau_mv_refresh.sql` fait, juste après chaque refresh des MV, un `INSERT ... WHERE NOT EXISTS` depuis chaque MV\_\* vers la table référentielle correspondante. Les colonnes `IDENTITY` sont temporairement basculées en `GENERATED BY DEFAULT AS IDENTITY` pour autoriser l'insertion explicite des IDs maîtres venus de Cergy, ce qui garde les clés primaires alignées entre les deux sites. Du coup, les FK sur Pau pointent toujours vers une ligne référentielle existante, et on garde quand même la transparence des MV pour les lectures.

## 8 Optimisation : index, cluster, plans d'exécution

On utilise cinq familles d'accélérateurs sur le schéma : index B-Tree pour les jointures à forte sélectivité, index Bitmap pour les filtres à faible cardinalité, index composites (par colonnes) pour les combinaisons fréquentes, index par fonction pour les expressions calculées, et cluster Oracle pour les jointures maître-détails systématiques. Tous les index sont créés dans le tablespace dédié `TBS_INDEX`, pour séparer physiquement les pages d'index des pages de données et limiter la contention I/O.

### 8.1 Index B-Tree : accélérer les jointures sur clés étrangères

Les **index B-Tree** sont des arbres triés équilibrés. Ils sont efficaces sur les colonnes à **forte sélectivité**, typiquement les clés étrangères, qui prennent autant de valeurs distinctes qu'il y a de lignes dans la table maître. On en a posé quatre sur les FK les plus jointes :

| Index                       | Table.Colonne              | Cible benchmark        | Justification                                                              |
|-----------------------------|----------------------------|------------------------|----------------------------------------------------------------------------|
| IDX_ATTRIBUTION_MATERIEL    | ATTRIBUTION.materiel_id    | Q3, Q5                 | FK jointe systématiquement lors de la reconstruction d'historique matériel |
| IDX_ATTRIBUTION_UTILISATEUR | ATTRIBUTION.utilisateur_id | Recherches utilisateur | FK jointe pour le tableau de bord d'un utilisateur                         |
| IDX_MATERIEL_MODELE         | MATERIEL.modele_id         | Q5                     | Jointure tripartite avec MODELE et CONSTRUCTEUR pour l'inventaire enrichi  |
| IDX_PORT_EQUIPEMENT         | PORT_RESEAU.equipement_id  | Vue d'un switch        | Liste des 24 ports d'un équipement réseau                                  |

Sur de gros volumes, un B-Tree transforme un *Full Table Scan* en `INDEX RANGE SCAN` en  $O(\log n)$ , avec accès direct au ROWID après traversée de l'arbre. Sur nos volumes modestes (quelques milliers de lignes), l'effet est partiellement masqué par le *Buffer Pool* qui garde toute la table en mémoire (voir l'analyse en section 9.5). L'index reste quand même indispensable dès qu'on dépasse quelques milliers de lignes.

## 8.2 Index Bitmap : compresser les filtres à faible sélectivité

Un **index Bitmap** stocke un vecteur de bits par valeur distincte de la colonne indexée. Chaque position du vecteur correspond à une ligne et indique son appartenance ou non au groupe. Deux avantages : (i) la structure se compresse bien quand la cardinalité est faible et la table volumineuse, et (ii) le moteur peut combiner les bitmaps par opérations booléennes (AND, OR, NOT) directement en mémoire, avant de toucher aux blocs de la table. Le revers de la médaille : une modification peut reconstruire un gros segment de bitmap, ce qui rend les Bitmap mauvais sur des tables très write-intensive.

On a posé quatre Bitmap sur des colonnes dont la cardinalité est bornée par le métier :

| Index                  | Table.Colonne             | Val. distinctes | Cible benchmark        |
|------------------------|---------------------------|-----------------|------------------------|
| IDX_BM_MATERIEL_STATUT | MATERIEL.statut           | 5               | Q2, Q8                 |
| IDX_BM_MATERIEL_ETAT   | MATERIEL.etat             | 4               | Q2, Q6                 |
| IDX_BM_EQ_TYPE         | EQUIPEMENT_RESEAU.type_id | 5               | Filtre catégorie       |
| IDX_BM_USER_ACTIF      | UTILISATEUR.actif         | 2 (0 ou 1)      | Filtre actif / inactif |

Ces index sont particulièrement efficaces sur Q2 (filtrage `statut + etat + site_id`) où l'optimiseur intersecte les bitmaps avant d'aller lire la table. Q8 (`GROUP BY type / statut / site`) en profite aussi : la lecture compacte du bitmap permet de générer les buckets de regroupement sans parcourir tous les blocs.

## 8.3 Index composites : couvrir les combinaisons multi-critères

Quand une clause `WHERE` filtre sur plusieurs colonnes en même temps, un index composite peut être plus rentable que deux index séparés : il rend les ROWID en une seule descente d'arbre. On en a créé deux selon nos patterns de requêtes :

- **IDX\_MATERIEL\_SITE\_STATUT** (`site_id`, `statut`) : pour les requêtes du type « matériels en service à Cergy ». On met `site_id` en tête car c'est la colonne la plus discriminante (deux moitiés équilibrées) ; `statut` affine ensuite.
- **IDX\_ATTRIBUTION\_USER\_DATEFIN** (`utilisateur_id`, `date_fin`) : pour retrouver les attributions actives (`date_fin IS NULL`) d'un utilisateur donné, typiquement la requête d'un tableau de bord. Oracle gère bien `IS NULL` en seconde colonne d'un index composite.

## 8.4 Index par fonction : recherches sur expression déterministe

Un **index par fonction** indexe le résultat d'une expression et non la valeur brute. Utile dès que la clause `WHERE` applique une transformation systématique sur la colonne, ce qui rendrait inutilisable un index classique. Condition essentielle : la fonction doit être **déterministe** (même entrée donne toujours la même sortie). `UPPER` l'est, `SYSDATE` ne l'est pas.

```
CREATE INDEX IDX_USER_UPPER_LOGIN ON UTILISATEUR (UPPER(login)) TABLESPACE TBS_INDEX;
```

Cet index est utilisé dès qu'une requête contient `WHERE UPPER(login) = UPPER(...)`, ce qu'on retrouve partout pour la recherche insensible à la casse (l'utilisateur peut taper `Jean.Martin1`, `JEAN.MARTIN1` ou `jean.martin1`, ça donne le même résultat). Sans l'index, le moteur fait un *Full Table Scan* avec calcul de `UPPER` ligne par ligne ; avec, la traversée se fait en  $O(\log n)$  sur les valeurs déjà normalisées.

## 8.5 Cluster CL\_MATERIEL\_ATTRIBUTION : cohabitation physique maître-détails

Un **cluster Oracle** regroupe sur les **mêmes blocs physiques** les lignes de plusieurs tables qui partagent une colonne commune (la *cluster key*). C'est utile pour les jointures maître-détails systématiquement faites ensemble : quand le moteur lit un matériel, il récupère dans le même bloc toutes ses attributions, ce qui économise les I/O qu'un accès séparé aux deux tables imposerait.

```
CREATE CLUSTER CL_MATERIEL_ATTRIBUTION (materiel_id NUMBER)
  SIZE 512 TABLESPACE TBS_MATERIELS;

CREATE TABLE MATERIEL      (...) CLUSTER CL_MATERIEL_ATTRIBUTION (id);
CREATE TABLE ATTRIBUTION  (...) CLUSTER CL_MATERIEL_ATTRIBUTION (materiel_id);

CREATE INDEX IDX_CL_MATERIEL_ATTRIBUTION ON CLUSTER CL_MATERIEL_ATTRIBUTION
  TABLESPACE TBS_INDEX;
```

Le `SIZE 512` indique la taille estimée d'un groupe de lignes partageant le même `materiel_id` (un matériel a généralement quelques attributions au fil du temps). L'**index de cluster** sur la clé est **obligatoire avant toute insertion** dans les tables clusterisées : c'est lui qui permet au moteur de localiser le bon bloc lors d'un `INSERT`. La jointure `MATERIEL JOIN ATTRIBUTION ON materiel_id` peut alors lire les deux côtés en un seul accès disque, ce qui devient un vrai gain sur de gros volumes.

## 8.6 Mise en cluster d'une table existante : protocole de migration

Inconvénient du cluster : on ne peut pas l'appliquer à une table existante directement, la clause `CLUSTER` doit être spécifiée au `CREATE TABLE`. `MATERIEL` et `ATTRIBUTION` étant déjà peuplées et liées par des FK, leur migration suit ce pattern en sept étapes :

1. **Sauvegarde** des données dans des tables temporaires (`CREATE TABLE temp_materiel AS SELECT * FROM MATERIEL`).
2. **Suppression** des tables originales avec `DROP TABLE ... CASCADE CONSTRAINTS`, qui retire automatiquement les FK pointant vers ces tables.
3. **Création du cluster** avec ses paramètres.
4. **Création des deux tables dans le cluster** via la clause `CLUSTER nom_cluster (colonne_correspondante)`.
5. **Création de l'index de cluster**. Obligatoire avant toute insertion.
6. **Restauration** des données depuis les tables temporaires.
7. **Recréation** de toutes les contraintes (FK, CHECK, UNIQUE) et des index secondaires sur les deux tables, puis suppression des tables temporaires.

À ce protocole on a dû ajouter une étape supplémentaire pour notre projet : réattribuer les `GRANT SELECT` à `link_user` sur les tables recrées, car le `DROP TABLE` a aussi effacé tous les privilèges. Sans ça, les vues globales distribuées (`V_MATERIELS_GLOBAL`) deviennent invalides avec un `ORA-04063: view has errors` au prochain compile, et toutes les requêtes inter-sites cassent en cascade.

# 9 Protocole de test et résultats

## 9.1 Volumétrie et stratégie de génération des jeux de données

La validation d'une architecture distribuée et fragmentée exige un jeu de données robuste, capable de simuler les contraintes de charge réelles d'une infrastructure d'entreprise. Pour ce faire, les

données opérationnelles ont été réparties de manière asymétrique entre le nœud central (**Cergy**) et le nœud régional (**Pau**).

Le site de Pau a été calibré de manière intentionnelle pour représenter environ **60 % du volume de données opérationnelles de Cergy**. Cette asymétrie reflète fidèlement la topologie d'un système d'information réel où le siège social concentre une densité d'actifs supérieure à celle d'une agence régionale, permettant ainsi de tester la base distribuée dans un contexte de fragments hétérogènes.

La volumétrie effective du parc interconnecté se répartit comme suit (exprimée en nombre de lignes physiques présentes et synchronisées sur chaque nœud) :

| Table                    | Cergy (Site 1) | Pau (Site 2)              | Nature de la donnée              |
|--------------------------|----------------|---------------------------|----------------------------------|
| <b>SITE</b>              | 2              | 2 (Répliqués via MV)      | Référentiel global               |
| <b>LOCATION</b>          | <b>9</b>       | <b>9</b> (Synchro locale) | Référentiel global               |
| <b>UTILISATEUR</b>       | 2 500          | 1 500                     | Donnée opérationnelle fragmentée |
| <b>MATERIEL</b>          | 5 000          | 3 000 (9 000 en charge)   | Donnée opérationnelle fragmentée |
| <b>EQUIPEMENT_RESEAU</b> | 500            | —                         | Donnée opérationnelle fragmentée |
| <b>PORT_RESEAU</b>       | 12 000         | —                         | Donnée opérationnelle fragmentée |
| <b>ATTRIBUTION</b>       | 5 000          | —                         | Donnée opérationnelle fragmentée |
| <b>VLAN</b>              | 5              | 4                         | Donnée opérationnelle fragmentée |
| <b>JOURNAL_AUDIT</b>     | ~13 000        | Variable (triggers)       | Audit local append-only          |

### 9.1.1 Ingénierie de génération des données

Afin d'éviter tout recours à des outils tiers, l'intégralité des injections de données de test a été codée en PL/SQL natif, en respectant deux contraintes techniques avancées :

1. **Génération déterministe et unique des adresses MAC et IP** : Pour insérer massivement 500 équipements réseau sans lever d'exceptions d'unicité, un algorithme découpant l'index de la boucle interne (*i*) sur deux octets distincts a été implémenté.
  - *Pour les adresses MAC* : La fonction isole la partie haute via un `TRUNC((i-1)/256)` et la partie basse via un `MOD(i-1, 256)`, garantissant des paires hexadécimales uniques jusqu'à 65 536 lignes sans risque de collision.
  - *Pour les adresses IP* : Une logique similaire applique une division par /250 pour formater dynamiquement des adresses de sous-réseau strictes (ex : 10.1.x.y).
2. **Synchronisation des clés primaires distantes (Nœud Pau)** : Sur le site de Pau,

les tables de référence locales doivent satisfaire des contraintes de clés étrangères (FK) pointant vers les vues matérialisées synchronisées depuis Cergy. Pour maintenir une parfaite symétrie des identifiants (IDs) sans altérer les séquences automatiques, les colonnes configurées en `GENERATED ALWAYS AS IDENTITY` subissent une modification temporaire en `GENERATED BY DEFAULT AS IDENTITY` lors de la phase de réplication. Cela permet d'injecter manuellement les identifiants maîtres d'origine avant de restaurer la contrainte nominale.

## 9.2 Algorithme du générateur paramétrique (`peupler_tout`)

Afin de valider la scalabilité verticale de notre schéma et de mesurer l'efficacité des index sur des volumétries variables, un package de génération paramétrique nommé `peupler_tout(p_ratio)` a été déployé sur les deux nœuds.

L'architecture de cette procédure repose sur les principes suivants :

- **Auto-détection dynamique du site d'exécution** : Le script interroge dynamiquement la table locale `SITE` (`SELECT code INTO v_prefix FROM SITE WHERE ROWNUM = 1`). En cas d'exécution sur le nœud secondaire de Pau (où la table locale est initialement vide), un bloc d'interception d'exceptions (`EXCEPTION WHEN OTHERS THEN`) bascule automatiquement la lecture sur la vue matérialisée `MV_SITE`.
- **Dimensionnement à l'échelle** : L'argument `p_ratio` fait office de multiplicateur de charge. À `p_ratio = 1`, les volumes nominaux sont appliqués. À `p_ratio = 5`, le générateur injecte automatiquement un volume modifié par 5 (~500 000 lignes cumulées), simulant un stress-test de l'infrastructure afin de pousser le moteur d'optimisation d'Oracle dans ses retranchements.

## 9.3 Protocole expérimental du benchmark

L'évaluation des performances s'articule autour de **8 requêtes SQL critiques**, exécutées à travers deux scénarios distincts :

1. **Scénario SANS\_INDEX** : Base de données brute, exempte de tout index secondaire ou de cluster physique.
2. **Scénario AVEC\_INDEX\_ALL** : Activation conjointe de l'ensemble des index B-Tree, index Bitmap, index composites, index fonctionnels et du cluster physique `CL_MATERIEL_ATTRIBUTION`.

### 9.3.1 Élimination du biais de Cold Cache

L'évaluation brute de requêtes SQL souffre généralement du biais de « Cold Cache », lié au temps d'accès initial aux blocs de données sur le disque dur. Pour s'assurer que le benchmark mesure rigoureusement la performance structurelle de l'indexation (algorithme de parcours) et non la performance des sous-systèmes d'I/O disques, le protocole applique une règle stricte de **triple exécution successive pour chaque requête**.

- La première exécution (Cold Cache) est écartée de la moyenne.
- Les deux exécutions suivantes sont réalisées en **Warm Cache**, c'est-à-dire une fois que le sous-ensemble de pages de données réside intégralement dans la mémoire vive du *Buffer Pool* d'Oracle.

La capture temporelle est opérée par l'API native `DBMS_UTILITY.GET_TIME`, mesurant le temps CPU de traitement à une résolution de l'ordre du centième de seconde (10 ms).



### 9.3.2 Cartographie des 8 requêtes de test

- **Q1 (Recherche unitaire par Numéro de Série)** : Filtre d'égalité prévenant les arrêts cardiaques d'Oracle via une clause robuste (`SELECT COUNT(*)`) afin de mesurer le coût du scan sans index.
- **Q2 (Filtre Multi-critères local)** : Requête combinant `statut + etat + site_id` pour mesurer l'efficacité des index Bitmaps combinés aux index composites.
- **Q3 (Jointure Historique globale)** : Jointure lourde `MATERIEL JOIN ATTRIBUTION` visant à valider le gain d'accès physique du cluster maître-détails.
- **Q4 (Recherche textuelle de login)** : Filtre utilisant `UPPER(login)` pour valider l'index basé sur une fonction (*Function-Based Index*) rendu robuste via un comptage.
- **Q5 (Jointure de référentiels)** : Jointure locale entre la table des matériels et les catalogues constructeurs/modèles.
- **Q6 (Requête analytique distribuée via Vue Globale)** : Interrogation de la vue `V_MATERIELS_GLOBAL` unifiée avec transit de données distantes via DB Link pour évaluer le coût du réseau.
- **Q7 (Lecture du Référentiel régional)** : Comparaison de coût entre un accès local sur la Vue Matérialisée de Pau versus un appel synchrone distant via DB Link vers Cergy.
- **Q8 (Agrégat de masse complet)** : Tri, groupement (`GROUP BY`) et calculs statistiques sur l'intégralité du volume matériel.

## 9.4 Résultats bruts du Benchmark

### 9.4.1 Résultats mesurés sur le nœud Cergy (Volume : 5 000 Matériels)

| Requête   | SANS_INDEX | AVEC_INDEX_ALL | Delta (Gain net)                      |
|-----------|------------|----------------|---------------------------------------|
| <b>Q1</b> | 0 ms       | 10 ms          | Bruit de mesure                       |
| <b>Q2</b> | 20 ms      | 0 ms           | <b>-20 ms (Gain de 100 %)</b>         |
| <b>Q3</b> | 10 ms      | 40 ms          | +30 ms (Surcharge structur-<br>relle) |
| <b>Q4</b> | 10 ms      | 10 ms          | stable                                |
| <b>Q5</b> | 10 ms      | 10 ms          | stable                                |
| <b>Q6</b> | 100 ms     | 90 ms          | -10 ms (Coût réseau domi-<br>nant)    |
| <b>Q8</b> | 40 ms      | 70 ms          | +30 ms                                |

### 9.4.2 Résultats effectifs mesurés sur le nœud Pau (~9 000 Matériels avec charge)

| Requête   | SANS_INDEX | AVEC_INDEX_ALL | Delta (Gain net)              |
|-----------|------------|----------------|-------------------------------|
| <b>Q1</b> | 20 ms      | 0 ms           | <b>-20 ms</b>                 |
| <b>Q2</b> | 70 ms      | 0 ms           | <b>-70 ms (Gain Critique)</b> |
| <b>Q3</b> | 40 ms      | 10 ms          | <b>-30 ms</b>                 |
| <b>Q4</b> | 30 ms      | 10 ms          | <b>-20 ms</b>                 |
| <b>Q5</b> | 0 ms       | 0 ms           | stable                        |
| <b>Q6</b> | 20 ms      | 80 ms          | Coût réseau asynchrone        |
| <b>Q7</b> | 0 ms       | 0 ms           | Instantané (MView locale)     |
| <b>Q8</b> | 70 ms      | 30 ms          | <b>-40 ms (Gain de 57 %)</b>  |

## 9.5 Analyse critique des performances et justifications des écarts

L'analyse des métriques met en évidence une adéquation claire entre la théorie des bases de données et les réalités physiques du moteur d'exécution d'Oracle :

1. **L'efficacité indiscutable des index Bitmap et composites (Q2 et Q8) :** Le gain massif enregistré sur la **Q2 de Pau (70 ms à vide)** et de **20 ms sur Cergy** illustre la puissance des structures de Bitmaps sur des colonnes à faible cardinalité (telles que l'état ou le statut du matériel). Le moteur d'Oracle parvient à exécuter des opérations booléennes (intersections de bits) directement en mémoire vive sur les vecteurs de bits avant même d'effectuer la moindre lecture physique de lignes dans les blocs de la table, accélérant drastiquement le filtrage et l'agrégation.
2. **Limites de la granularité temporelle et du volume de données (Q5) :** Certaines requêtes affichent un score persistant de 0 ms. Ce phénomène s'explique par deux facteurs de contournement :
  - La granularité de l'horloge système de `DBMS_UTILITY.GET_TIME` est limitée au centième de seconde (10 ms). Toute exécution ultra-rapide sub-10 ms est arrondie par défaut à 0 ms.
  - Notre volume nominal permet au moteur d'exécuter un *Full Table Scan* presque instantanément si les pages sont déjà chargées dans le *Buffer Pool* (Warm Cache), masquant l'apport algorithmique théorique des index sur de très petites tables référentielles.
3. **Justification du comportement des jointures de cluster (Q3) :** On observe un écart comportemental net entre Cergy (+30 ms de surcharge) et Pau (-30 ms de gain net après indexation). Cette divergence est un comportement classique documenté : l'organisation physique en cluster impose une surcharge d'encapsulation (chaînage des blocs, index de cluster obligatoire). Sur de petits volumes, la surconsommation CPU nécessaire pour traverser l'index du cluster est supérieure à un simple balayage séquentiel direct en mémoire, tandis que sur le volume rehaussé de Pau, l'économie d'I/O physiques commence à amortir structurellement cette friction.
4. **Prédominance du coût réseau dans l'architecture distribuée (Q6) :** La requête globale unifiée, stabilisée entre 20 ms (cache optimisé par UNION ALL disjoint) et 80–100 ms selon la charge, démontre de manière empirique que la latence réseau induite par le Database Link (sérialisation des paquets, transport via Oracle Net, désérialisation sur le nœud distant) est le facteur limitant absolu. Le coût d'acheminement des blocs à travers le réseau Docker est d'un ordre de grandeur supérieur au coût de traitement local des lignes par le CPU.

## 9.6 Validation de l'architecture distribuée : Matrice MV vs DB Link distant

L'un des résultats les plus formateurs du benchmark réside dans la validation empirique de la stratégie de réplication choisie pour les référentiels globaux (Requête Q7).

Le tableau ci-dessous isole le coût d'accès à la table de référence `SITE` selon la méthode d'interrogation choisie depuis le nœud régional de Pau :

| Méthode d'accès depuis Pau                                             | Latence    | Impact sur l'infrastructure                                  |
|------------------------------------------------------------------------|------------|--------------------------------------------------------------|
| <b>DB Link synchrone distant</b><br>( <code>SITE@dblink_cergy</code> ) | ~80–100 ms | Consommation de bande passante, dépendance à Cergy           |
| <b>Vue matérialisée locale</b><br>( <code>MV_SITE</code> )             | 0–10 ms    | Lecture locale instantanée, isolation en cas de panne réseau |

**Conclusion de l'évaluation BDDR :** L'utilisation de Vues Matérialisées locales permet de diviser par un facteur 100 le temps d'accès aux tables de référence, validant la pertinence

de notre architecture hybride. L'accès distant par Database Link doit être strictement circonscrit aux transactions d'écriture distribuées (ex : transfert inter-sites de matériel informatique via PKG\_ADMIN\_PARC) et aux processus planifiés asynchrones de rafraîchissement des logs (REFRESH FAST).

## 10 Architecture des fichiers et exécution

### 10.1 Arborescence du dépôt

```
/docker/      -> docker-compose.yml + scripts init
/schema/      -> DDL : tablespaces, owner, tables, contraintes, users/roles
/plsql/       -> Packages, triggers, curseur batch
/perf/        -> MV logs, db links, MV, vues globales, index, cluster, benchmarks
/tests/       -> Referentiels, donnees operationnelles, generateur, benchmark baseline
init.sh       -> Orchestrateur qui rejoue tout dans l'ordre
sujet.pdf     -> Le sujet initial
rapport.pdf   -> Le présent document
README.md     -> Instructions d'utilisation et commandes de lancement
```

### 10.2 Reproduction complète

```
# 1. Lancer les deux conteneurs
docker compose -f docker/docker-compose.yml up -d

# 2. Attendre que Oracle soit pret (~60 s) puis executer l'orchestrateur
bash init.sh

# 3. Mesures avant index
sqlplus GLPI_OWNER/admin123@localhost:1521/XEPDB1 @tests/06_benchmark_baseline.sql
sqlplus GLPI_OWNER/admin123@localhost:1522/XEPDB1 @tests/06_benchmark_baseline.sql

# 4. Application des optimisations
for f in perf/05_indexes_btree.sql perf/06_indexes_bitmap.sql \
        perf/07_indexes_composite.sql perf/08_index_functionbased.sql \
        perf/09_cluster_materiel_attribution.sql; do
    sqlplus GLPI_OWNER/admin123@localhost:1521/XEPDB1 @$f
    sqlplus GLPI_OWNER/admin123@localhost:1522/XEPDB1 @$f
done

# 5. Mesures apres optimisation
sqlplus GLPI_OWNER/admin123@localhost:1521/XEPDB1 @perf/10_benchmark_post_index.sql
sqlplus GLPI_OWNER/admin123@localhost:1522/XEPDB1 @perf/10_benchmark_post_index.sql
```

## 11 Limites et axes d'amélioration

### 11.1 Limites du schéma et de l'infrastructure

- **Matrice rôles à étendre** pour un passage production : profils plus granulaires (lecture seule par site, opérateur niveau 1 vs niveau 2, etc.).
- **Oracle XE en Docker** présente des limites inhérentes au contexte pédagogique : limites mémoire, parallélisme restreint ...

### 11.2 Limites de la logique métier

- Le **journal d'audit** sérialise les lignes en **texte simple** ; un format JSON aurait été plus structuré et facilement requêtable.

- Les triggers d’audit couvrent **les quatre tables principales**, mais pas toutes les tables du modèle (le réseau et les attributions associatives ne sont pas tracés).
- **recalculer\_statistiques\_parc** fait un **recalcul complet**, en production, on pourrait envisager un recalcul incrémental basé sur les MV logs.
- Le **transfert inter-sites ne gère pas de mécanisme applicatif de reprise** si le nœud distant est indisponible : la transaction distribuée Oracle est annulée, mais aucune retry ou file d’attente n’est implémentée.
- Les **erreurs métier sont centralisées** dans PKG\_EXCEPTIONS, mais pourraient être enrichies avec des **logs techniques plus détaillés** (contexte d’appel, valeurs en cause, stack trace).

### 11.3 Limites BDDR et performance

- **Volumétrie modeste** ( $\leq 12\,000$  lignes par table) qui ne met pas toujours en évidence l’avantage des accélérateurs. Sur des volumes de l’ordre du million, les gains attendus seraient plus francs.
- **Résolution de mesure** : DBMS\_UTILITY.GET\_TIME est exprimé en centièmes de seconde. Un appel  $< 10$  ms est mesuré à 0. Pour des mesures fines, il faudrait DBMS\_UTILITY.GET\_CPU\_TIME ou un échantillonnage répété.
- **Pas de chiffrement** des communications entre Cergy et Pau : le mot de passe `link_user` circule en clair dans la chaîne TNS.
- **Pas de gestion de panne** : si Cergy est indisponible, les vues globales sur Pau échouent. Un mécanisme de fallback ou de cache local étendu serait nécessaire.

### 11.4 Pistes d’extension

- Partitionnement par site (Oracle PARTITION BY LIST) pour combiner fragmentation logique et partitionnement physique.
- Migration vers REFRESH FAST complet en s’assurant que tous les MV logs supportent le mode (certaines colonnes peuvent forcer un fallback COMPLETE).
- Mise en place d’un journal de réplication asynchrone pour les opérations cross-site (au-delà du transfert de matériel).
- Migration du journal d’audit vers un format JSON et indexation par opération/table.

Ces limites sont acceptables dans le cadre du projet, car l’objectif principal était de démontrer l’utilisation correcte des notions Oracle enseignées : tablespaces, rôles, PL/SQL, BDDR, optimisation par index et clusters.

## 12 Conclusion

Au terme du projet, nous avons une base Oracle distribuée fonctionnelle qui couvre toutes les notions abordées en cours. Le schéma à 17 tables est issu d’un reverse engineering de GLPI ; il repose sur 6 tablespaces séparés et 5 rôles applicatifs construits sur un schéma propriétaire GLPI\_OWNER. La logique métier est entièrement écrite en PL/SQL : packages d’exceptions, fonctions de lecture, procédures d’administration, triggers d’intégrité et d’audit, et une procédure batch à curseur explicite. Côté BDDR, les données opérationnelles sont fragmentées horizontalement par `site_id`, les référentiels sont répliqués sur Pau via huit vues matérialisées rafraîchies par un job programmé, et trois vues globales en UNION ALL consolident les fragments des deux sites. Enfin, les optimisations (index B-Tree, Bitmap, composites, function-based et cluster) ont été appliquées et mesurées avec un protocole de benchmark avant/après.

Le tout est rejouable sur un environnement vierge avec `docker compose up -d` puis `bash init.sh`.

Sur le plan des résultats mesurés, on retrouve bien ce qui était attendu d'après les cours : les index Bitmap font gagner du temps sur les filtres à faible cardinalité (Q2 et Q8 surtout), le cluster ne donne pas vraiment de gain à notre échelle de quelques milliers de lignes, et la vue matérialisée locale lit un référentiel environ 100 fois plus vite qu'un accès distant via db link.

Travailler à quatre sur un même schéma nous a aussi obligés à bien découper le travail. La procédure de transfert inter-sites est un bon exemple : elle est écrite en PL/SQL mais dépend des db links pour s'exécuter. L'astuce du `EXECUTE IMMEDIATE` nous a permis de compiler le package sans attendre que les liens soient créés, et d'avancer en parallèle.